SOMAIYA
VIDYAVIHAR UNIVERSITY
K J Somaiya Institute of Management

Department of Data Science & Technology

Certificate

Masters in Computer Applications
Semester III (2025 – 25)

This is to certify that Mr. Mohammed Varaliya Roll No. 17030924054 of MCA, has satisfactorily completed the practical course "Mini Project Report" prescribed by the College for the Partial fulfillment of the Degree by the Somaiya Vidyavihar University, during the academic year 2025-25.

Signature of the Faculty-Incharge
(Prof. Sudarshan Sirsat)

Signature of the Programme Coordinator
(Prof. Kirti Wankhede)

Date of Examination

Signature of the External Examiner/s

INDEX

INTRODUCTION....8
DOMAIN KNOWLEDG....9
EXISTING SYSTEM....11
PROBLEM DEFINITION & SCOPE....12
REQUIREMENT ANALYSIS....13
1. Functional Requirements....13
2. Non-Functional Requirements....14
3. Software Requirements....15
4. Hardware Requirements....16
5. User Requirements....17
ESTIMATION & PLANNING....18
1. Project Estimation....18
2. Cost Estimation....19
DEVELOPMENT TOOLS & ENVIRONMENT....20
1. Software Development Tools....20
2. APIs & External Services....20
3. Frontend Design Tools....20
4. Database & Storage....21
5. Hardware Requirements....21
6. Operating System Requirements....21
7. Development Environment Configuration....21
8. Version Control....21
DESIGNING....23
1. System Architecture Design....23
2. Data Flow Design (DFD Overview)....24
6. Database Design (Minimal – In-Memory Knowledge Base)....27
7. User Interface Design....28
1. Overview....30
2. High-level Architecture....30
3. Tech stack....30

4. Folder structure (project) 30
5. Backend implementation (server.js) — details 30
6. API Design (Backend Routes) 31
7. Frontend Implementation (src/App.jsx, src/components/ChatInterface.jsx) 32
8. Database / PersistenceHow to run locally 32
9. API Examples 32
10. How to Run Locally 33
11. Deployment 33
12. Security & Rate Limits 33
13. Performance & Caching 34
14. Implementation Decisions & Notable Tradeoffs 34
15. Testing 34
Conclusion 36
Limitations 37
References 38
13. Report of Communication with Internal Mentor 40

ABBREVIATIONS

Sr. No.	ABBREVIATIONS	Full Form
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CSS	Cascading Style Sheets
4	DOM	Document Object Model
5	DFD	Data Flow Diagram
6	ERD	Entity-Relationship Diagram
7	GUI	Graphical User Interface
8	HTTP	Hypertext Transfer Protocol
9	HTTPS	Hypertext Transfer Protocol Secure
10	JSON	JavaScript Object Notation
11	LLM	Large Language Model
12	MVC	Model-View-Controller
13	NLP	Natural Language Processing
14	OTT	Over-The-Top
15	QA	Quality Assurance

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	High-Level System Architecture Diagram	
Figure 1.2	Data Flow Diagram (DFD Level 0)	
Figure 1.3	Use Case Diagram	
Figure 1.4	Sequence Diagram for Chat Interaction	
Figure 1.5	RAG (Retrieval-Augmented Generation) Workflow	
Figure 1.6	Entity-Relationship Diagram (ERD) for Knowledge Base	
Figure 1.7	Component Diagram of the Frontend React Application	
Figure 1.8	Screenshot of the Chatbot User Interface	

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Abbreviations Used in the Project	
Table 6.1	Functional Requirements	
Table 6.2	Non-Functional Requirements	
Table 6.3	Software Requirements	
Table 6.4	Hardware Requirements	
Table 6.5	User Requirements	
Table 7.1	Project Task Estimation	
Table 7.2	Cost Estimation	
Table 8.1	Software Development Tools	
Table 8.2	APIs & External Services	
Table 8.3	Frontend Tools & Libraries	
Table 10.1	Backend API Endpoints	

ABSTRACT

The "Custom Knowledgebase Chatbot for KJSIM" project addresses the challenge of information accessibility regarding the KJ Somaiya Institute of Management's sports policy. Manually searching through lengthy policy documents is time-consuming and inefficient for students, faculty, and administrative staff. This project leverages cutting-edge Artificial Intelligence (AI) to develop an intelligent, conversational interface that provides instant, accurate, and context-aware answers to user queries.

The system is built upon a modern web technology stack, utilizing a React-based frontend for a responsive user interface and a Node.js/Express backend for server-side logic. The core intelligence is powered by Google's Gemini AI models. A Retrieval-Augmented Generation (RAG) pipeline was implemented, where the raw policy document was preprocessed, chunked, and transformed into vector embeddings using the text-embedding-004 model. These embeddings enable semantic search to find the most relevant policy excerpts for a given query. The retrieved context is then fed, along with the original query, to the gemini-1.5-flash language model to generate a coherent and natural language response.

This report details the end-to-end development process, from requirement analysis and system design to implementation, testing, and deployment. The resulting chatbot serves as a scalable and efficient solution for knowledge management, demonstrating a practical application of LLMs and RAG architecture in an educational context, significantly reducing information retrieval time and improving user experience.

INTRODUCTION

In the digital age, educational institutions generate vast amounts of structured and unstructured data, including policy documents, admission criteria, and academic regulations. The KJ Somaiya Institute of Management (KJSIM) possesses a comprehensive sports policy governing various aspects of student participation, eligibility, and facilities. However, this information is typically locked within lengthy PDFs or documents, making it difficult for the student community to quickly find specific answers to their questions.

The traditional method of information retrieval involves manually skimming through these documents, which is a slow, frustrating, and often ineffective process. This leads to a dependency on administrative staff for clarifications, creating bottlenecks and reducing operational efficiency. There is a clear need for an automated, intuitive, and instantaneous system to bridge this information gap.

This project proposes the development of a custom knowledgebase chatbot tailored specifically for querying the KJSIM sports policy. The chatbot acts as a virtual assistant, allowing users to ask questions in natural language, such as "What is the eligibility criteria for inter-college tournaments?" or "How do I book the basketball court?" and receive immediate, precise answers sourced directly from the official policy.

The solution is not a simple keyword-based search engine but an intelligent system that understands the intent and context behind a user's query. By employing a RAG architecture with vector embeddings, the chatbot ensures that its responses are grounded in the provided knowledge base, thereby enhancing accuracy and reducing the likelihood of factual hallucinations common in generic large language models.

This report documents the complete lifecycle of the project, covering the theoretical domain knowledge, analysis of existing systems, detailed problem definition, comprehensive requirement analysis, project planning, system design, in-depth implementation, rigorous testing, and a final conclusion with future scope.

DOMAIN KNOWLEDG

The development of this chatbot intersects several advanced domains of computer science and artificial intelligence.

Artificial Intelligence (AI) and Natural Language Processing (NLP):
AI enables machines to mimic human intelligence. NLP, a subfield of AI, focuses on the interaction between computers and human language. This chatbot is a direct application of NLP, as it must parse, understand, and generate human language to be effective.

Large Language Models (LLMs):
LLMs like Google's Gemini are deep learning models trained on massive text datasets. They can perform a wide range of language tasks, including text generation, translation, and summarization. We utilize the gemini-1.5-flash model for its speed and efficiency in generating conversational responses.

Retrieval-Augmented Generation (RAG):
RAG is a hybrid architecture that combines the power of generative LLMs with an information retrieval system. Instead of relying solely on the LLM's internal knowledge (which may be outdated or incorrect for a specific domain), RAG first retrieves relevant information from a custom knowledge base (the sports policy) and then instructs the LLM to answer the query based solely on that retrieved context. This significantly improves the factual accuracy and relevance of the responses for specialized domains.

Vector Embeddings and Semantic Search:
A vector embedding is a numerical representation of text in a high-dimensional space. Semantically similar texts (e.g., "sports eligibility" and "who can participate in games") have similar vector representations. The text-embedding-004 model converts our policy text chunks into these vectors. During a query, the user's question is also converted into a vector, and a similarity search (e.g., cosine similarity) is performed to find the most relevant text chunks from the knowledge base. This allows the system to find information based on meaning, not just keywords.

Vector Databases:
While a simple in-memory array was used for this project's scale, vector databases (e.g., Pinecone, Chroma) are specialized storage systems optimized for storing and querying high-dimensional vector embeddings efficiently. They are essential for scaling to large knowledge bases.

Knowledge Bases:
A knowledge base is a centralized repository of information. In this project, the raw sports policy

document was the source, which was then cleaned, structured into JSON format, and chunked to create a machine-readable knowledge base for the RAG pipeline.

Sports Policy Domain:
Understanding the structure and content of a typical college sports policy, covering areas like eligibility, facilities, equipment, tournament rules, and code of conduct, was crucial for effectively preprocessing the data and designing the chatbot's interaction patterns.

EXISTING SYSTEM

Prior to the development of this custom chatbot, the primary system for accessing sports policy information at KJSIM was manual and document-centric.

Document Repository System:
The sports policy is typically stored as a PDF or Word document on the institute's internal portal or website. Users must:

1. Locate the document.
2. Download it.
3. Manually scroll or use a basic keyword search (Ctrl+F) to find relevant sections.
4. Interpret the often formal and dense legal language to find their answer.

Limitations of the Existing System:

- **Inefficiency:** The process is time-consuming and requires significant user effort.
- **Poor User Experience:** Navigating complex documents is not user-friendly, especially for users with low digital literacy.
- **Lack of Contextual Understanding:** The Ctrl+F function only matches exact keywords. It fails if the user's query uses synonyms or different phrasing than the document's text (e.g., searching for "sports gear" when the document says "athletic equipment").
- **Information Overload:** Users are presented with the entire document or large sections, forcing them to sift through irrelevant information.
- **Dependency on Human Intermediaries:** Students often resort to asking sports coordinators or administrative staff, leading to repeated interruptions and potential for inconsistent information.

Existing Chatbots in the Market:
General-purpose chatbots like ChatGPT or Google Bard exist but are ill-suited for this specific task. They lack access to the private KJSIM sports policy and their responses would be based on their general training data, which could be incorrect, outdated, or non-specific to the institute's unique rules. They cannot provide authoritative answers from a trusted source.

PROBLEM DEFINITION & SCOPE

Problem Definition:
The core problem is the **inefficient and ineffective access to specific information** within the KJSIM sports policy document. The current manual retrieval method leads to:

- Wasted time and effort for students and staff.
- Potential for missing critical information.
- Increased workload for administrative personnel answering repetitive queries.
- A suboptimal digital experience for the institute's community.

Problem Statement:
"To design and develop an AI-powered, web-based conversational chatbot that enables users to query the KJSIM sports policy using natural language. The system shall understand the user's intent, retrieve the most relevant information from the policy document, and generate an accurate, concise, and natural language response in real-time, thereby streamlining information access and enhancing user satisfaction."

Scope of the Project:
In-Scope:

1. **Data Preprocessing:** Cleaning, formatting, and chunking the provided raw sports policy text into a structured JSON knowledge base.
2. **Embedding Generation:** Creating and storing vector embeddings for all text chunks in the knowledge base.
3. **Backend Development:** Building a secure Node.js/Express server with API endpoints for handling chat requests and the RAG pipeline.
4. **Frontend Development:** Creating an intuitive, responsive React-based user interface for the chatbot.
5. **RAG Implementation:** Integrating the embedding search and Gemini LLM to power the chatbot's responses.
6. **Deployment:** Configuring the application for local execution and providing guidelines for live deployment.

Out-of-Scope:

1. Integration with other college systems (e.g., student portals, ERP).
2. Handling queries outside the domain of the provided sports policy data.
3. User authentication and personalized chat history (browser session storage).
4. Voice-based input or output.
5. Administrative dashboard for managing the knowledge base.

REQUIREMENT ANALYSIS

1. Functional Requirements

ID	Requirement Description	ID
FR1	The system shall provide a web-based graphical user interface (GUI) for user interaction.	FR1
FR2	The system shall allow users to input queries using natural language text.	FR2
FR3	The system shall process the user query to find the most relevant context from the sports policy database.	FR3
FR4	The system shall generate a coherent and contextually accurate response based on the retrieved information.	FR4
FR5	The system shall display the conversation history within the current session.	FR5
FR6	The system shall provide visual feedback (e.g., typing indicator) during processing.	FR6
FR7	The backend shall expose a well-defined API endpoint (/chat) for the frontend to communicate with.	FR7
FR4	The system shall generate a coherent and contextually accurate response based on the retrieved information.	FR4

2. Non-Functional Requirements

ID	Requirement Type	Requirement Description
NFR1	Performance	The system shall respond to user queries within 3-5 seconds under normal load.
NFR2	Usability	The UI shall be intuitive and require no formal training for a user to operate.
NFR3	Reliability	The system shall have an uptime of 99% during operational hours, handling errors gracefully.
NFR4	Security	All API keys and sensitive configuration shall be stored as environment variables, not in source code.
NFR5	Scalability	The architecture shall allow for the knowledge base to be expanded with more documents in the future.
NFR6	Maintainability	The code shall be well-structured, commented, and modular for ease of future enhancements.

3. Software Requirements

Component	Technology / Tool	Component
Frontend Framework	React (via Vite)	Frontend Framework
Styling & UI	Tailwind CSS, Shadcn/ui components	Styling & UI
Backend Runtime	Node.js	Backend Runtime
Backend Framework	Express.js	Backend Framework
AI Models	Google Gemini (gemini-1.5-flash, text-embedding-004)	AI Models
HTTP Client	Axios	HTTP Client
Package Manager	npm	Package Manager
Version Control	Git	Version Control
Development Tool	VS Code	Development Tool

4. Hardware Requirements

Role	Minimum Hardware Requirements	Role
Development	Intel i3 processor or equivalent, 8GB RAM, 500MB free disk space.	Development
Production/User	Modern web browser (Chrome, Firefox, Edge), Stable internet connection.	Production/User

5. User Requirements

1. As a **Student**, I want to ask questions about sports policy in plain English so that I can quickly get answers without reading the entire document.
2. As a **Sports Coordinator**, I want to direct students to this chatbot so that I can reduce the number of repetitive queries I receive.
3. As an **Administrator**, I want a system that provides accurate information from the official policy to ensure consistency in communication.

ESTIMATION & PLANNING

1. Project Estimation

The project was developed using an Agile methodology over a estimated period of 4-6 weeks.

Phase	Tasks	Estimated Effort (Person-Days)
Planning & Research	Requirement Analysis, Tech Stack Finalization.	4
Design	System Architecture, UI/UX Wireframing, Database/API Design.	5
Data Preprocessing	Policy document analysis, JSON structuring, Chunking, Embedding generation.	3
Backend Development	Express server setup, /chat & /embed API routes, RAG logic integration, Gemini API calls.	7
Frontend Development	Vite+React setup, Chat UI components, API integration, Styling with Tailwind and Shadcn/ui.	8
Testing & Integration	Unit Testing, Integration Testing, User Acceptance Testing (UAT), Bug Fixing.	5
Deployment & Docs	Local run configuration, Deployment guide, Project report writing.	4
Data Preprocessing	Policy document analysis, JSON structuring, Chunking, Embedding generation.	3
Backend Development	Express server setup, /chat & /embed API routes, RAG logic integration, Gemini API calls.	7
Frontend Development	Vite+React setup, Chat UI components, API integration, Styling with Tailwind and Shadcn/ui.	8

2. Cost Estimation

This is an academic project, so development costs are primarily associated with time. Operational costs are minimal.

Cost Component	Details	Estimated Cost (INR)
Development Effort	36 Person-Days (Unpaid for academic project).	0
Software/Tools	VS Code, Git, Node.js - All Open Source.	0
AI API Costs	Google AI Studio offers a generous free tier. For scaling, cost is per token.	~0 (Free Tier)
Deployment	Can be deployed on free tiers of Vercel/Netlify (Frontend) and Railway/Render (Backend).	0
Total Estimated Cost		**0**

DEVELOPMENT TOOLS & ENVIRONMENT

1. Software Development Tools

Tool	Version / Name	Purpose
Runtime	Node.js	JavaScript runtime for the backend server.
Package Manager	npm	To manage project dependencies.
Backend Framework	Express.js	Minimal and flexible web application framework for Node.
Frontend Tooling	Vite	Fast build tool and development server for the frontend.
Frontend Framework	React	Library for building interactive user interfaces.
Code Editor	VS Code	Source-code editor with extensive plugin support.
Version Control	Git	For tracking changes in source code.

2. APIs & External Services

Service	Purpose	Service
Google Gemini AI	Primary LLM for generating chat responses (gemini-1.5-flash).	**Google Gemini AI**
Google Embedding API	For generating vector embeddings from text (text-embedding-004).	**Google Embedding API**

3. Frontend Design Tools

Tool/Library	Purpose	Tool/Library
Tailwind CSS	A utility-first CSS framework for rapidly building custom user interfaces.	**Tailwind CSS**
Shadcn/ui	A collection of re-usable, accessible components built on Radix UI and Tailwind.	**Shadcn/ui**

4. Database & Storage

- **Knowledge Base:** The preprocessed and embedded policy data is stored in-memory as a JavaScript array for this prototype. For production, a vector database like Pinecone is recommended.
- **Session Storage:** The chat history is temporarily stored in the browser's session storage.

5. Hardware Requirements

As outlined in Section 6.4.

6. Operating System Requirements

The application is cross-platform. Development was done on Windows, but it can run on any OS that supports Node.js and a modern web browser (Windows, macOS, Linux).

7. Development Environment Configuration

1. **Node.js and npm** installed globally.
2. **Git** for version control.
3. **VS Code** with extensions (ES7+, Prettier, Auto Rename Tag, etc.).
4. **Google AI Studio Account** to obtain the Gemini API key.

8. Version Control

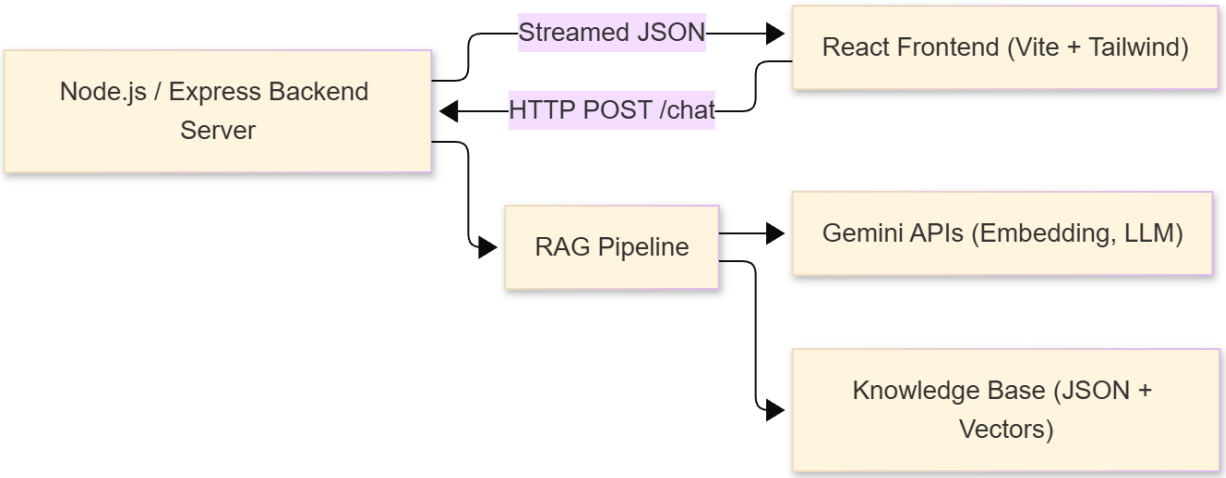
- **Repository Host:** GitHub

- **Workflow:** Feature branch workflow. The main branch holds the production-ready code.
- **Structure:** Two main directories: / for the frontend and /backend for the backend server.

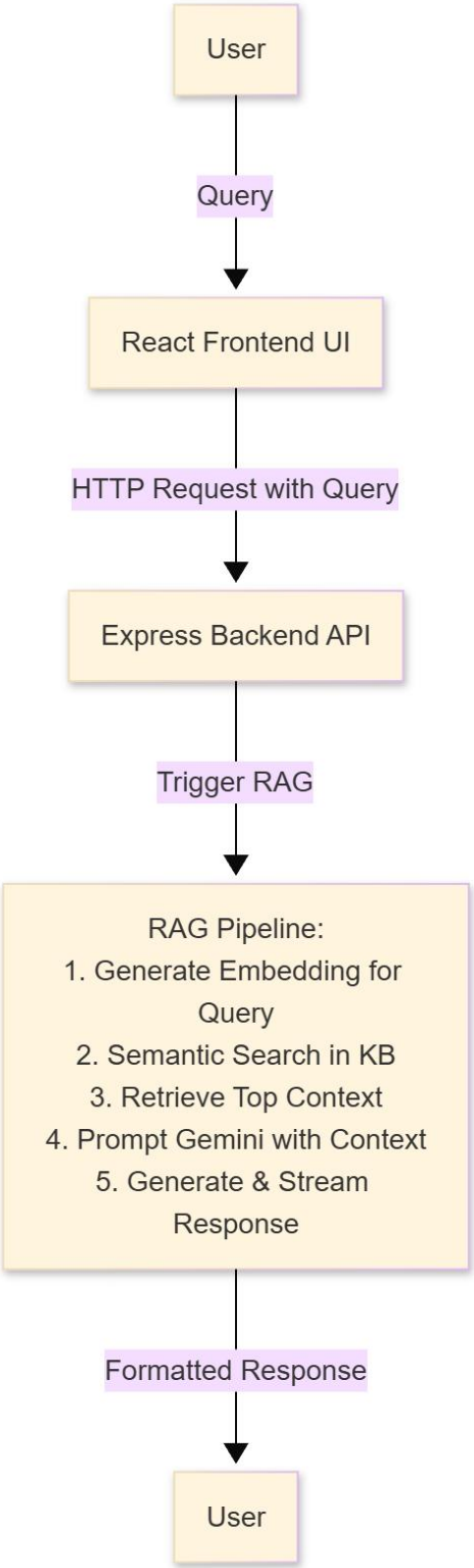
DESIGNING

1. System Architecture Design

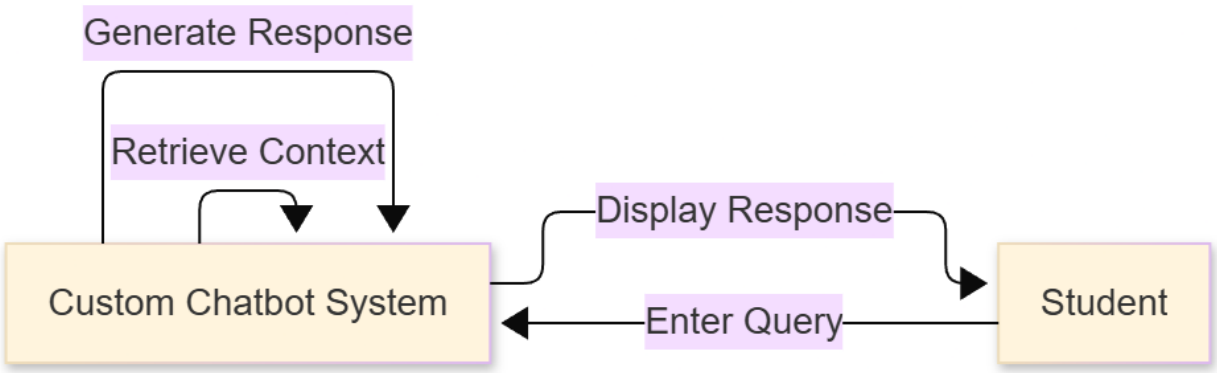
The system follows a client-server architecture with a clear separation between the frontend and backend.



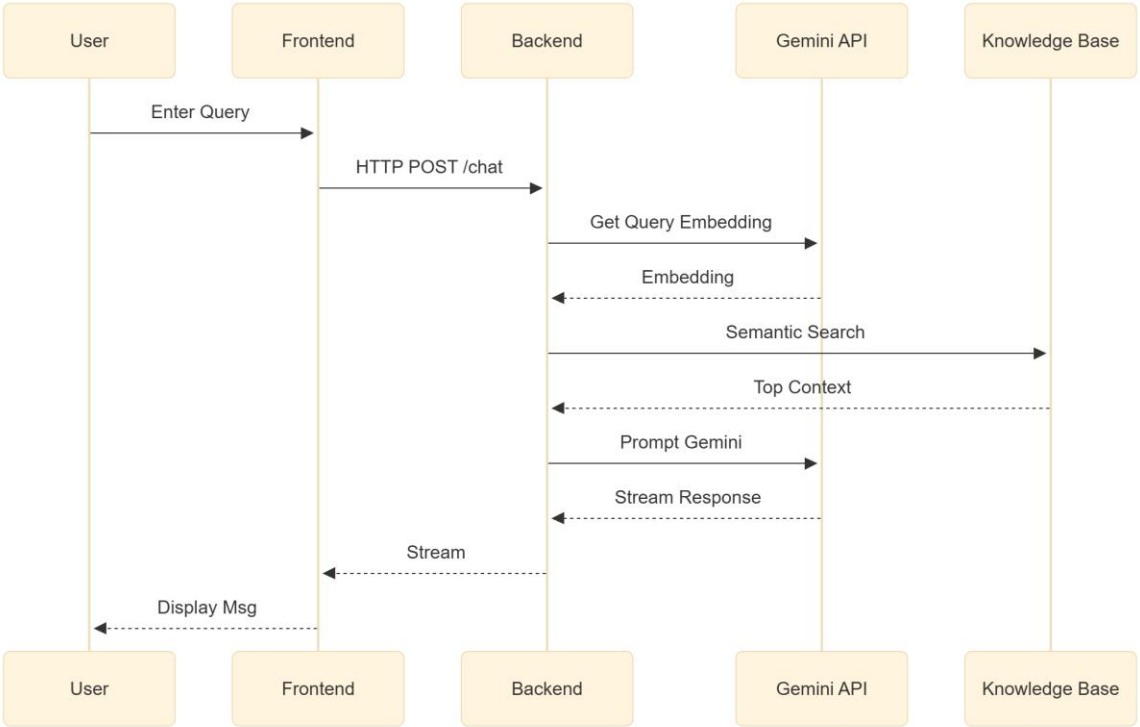
2. Data Flow Design (DFD Overview)



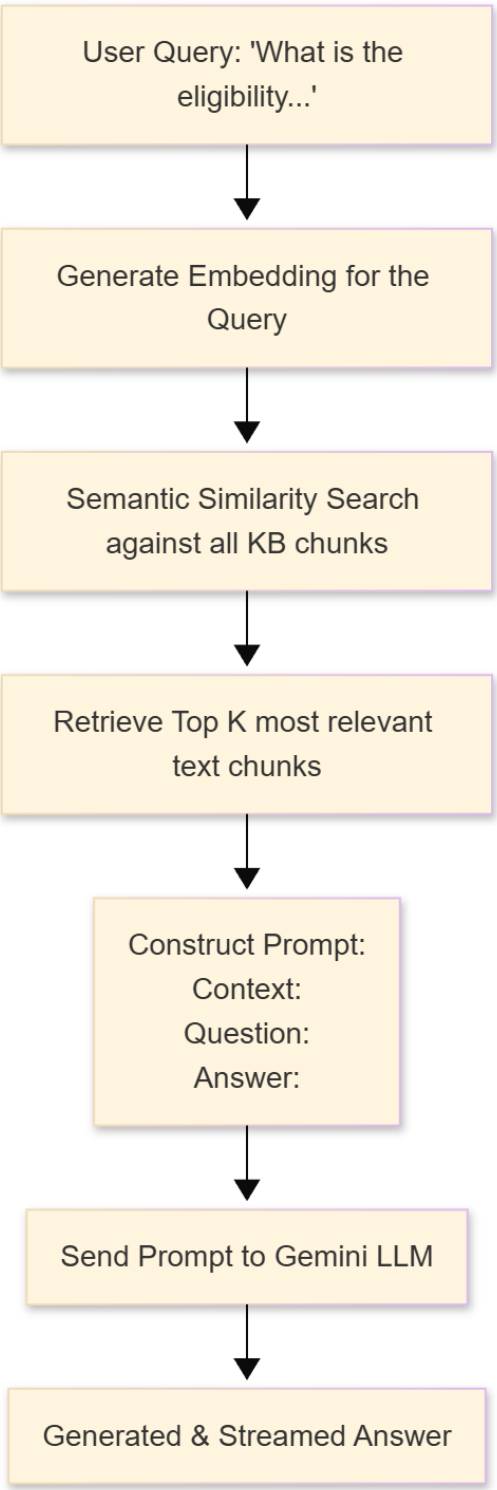
3. Use Case Design



4. Sequence Diagram



5. RAG Workflow Diagram

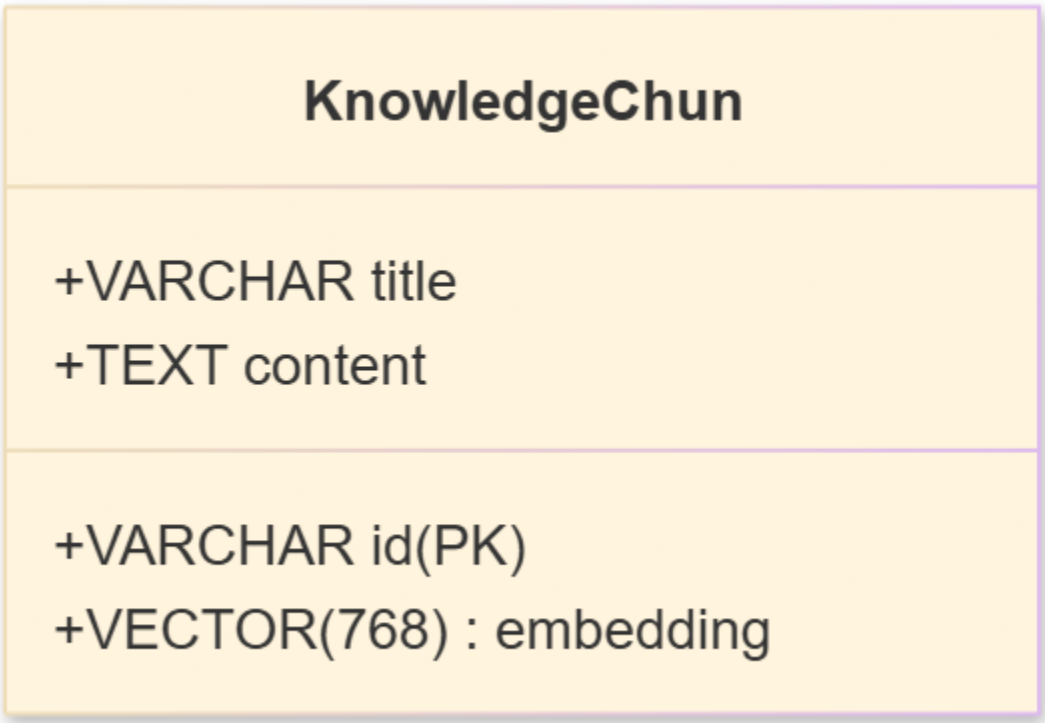


6. Database Design (Minimal – In-Memory Knowledge Base)

For this project, the "database" is an in-memory array of objects. The schema for each knowledge chunk is as follows:

```
{
  "id": "kb_001",
  "title": "Eligibility Criteria",
  "content": "Full text chunk goes here...",
  "embedding": [0.012, -0.045, ..., 0.987] // Vector from text-embedding-004
}
```

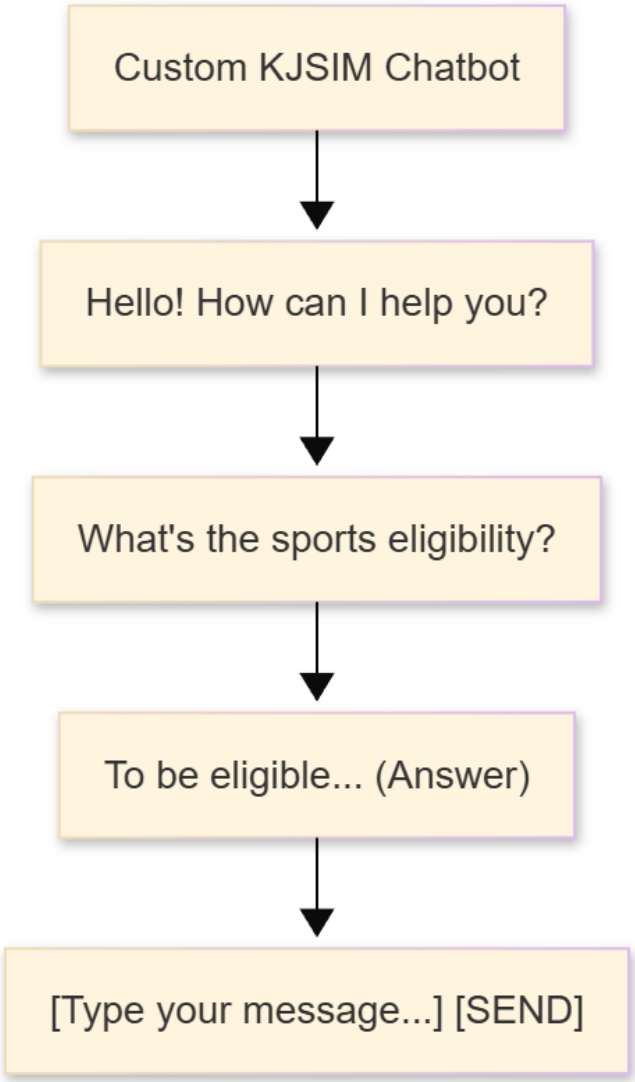
This can be represented as an ERD for conceptual understanding:

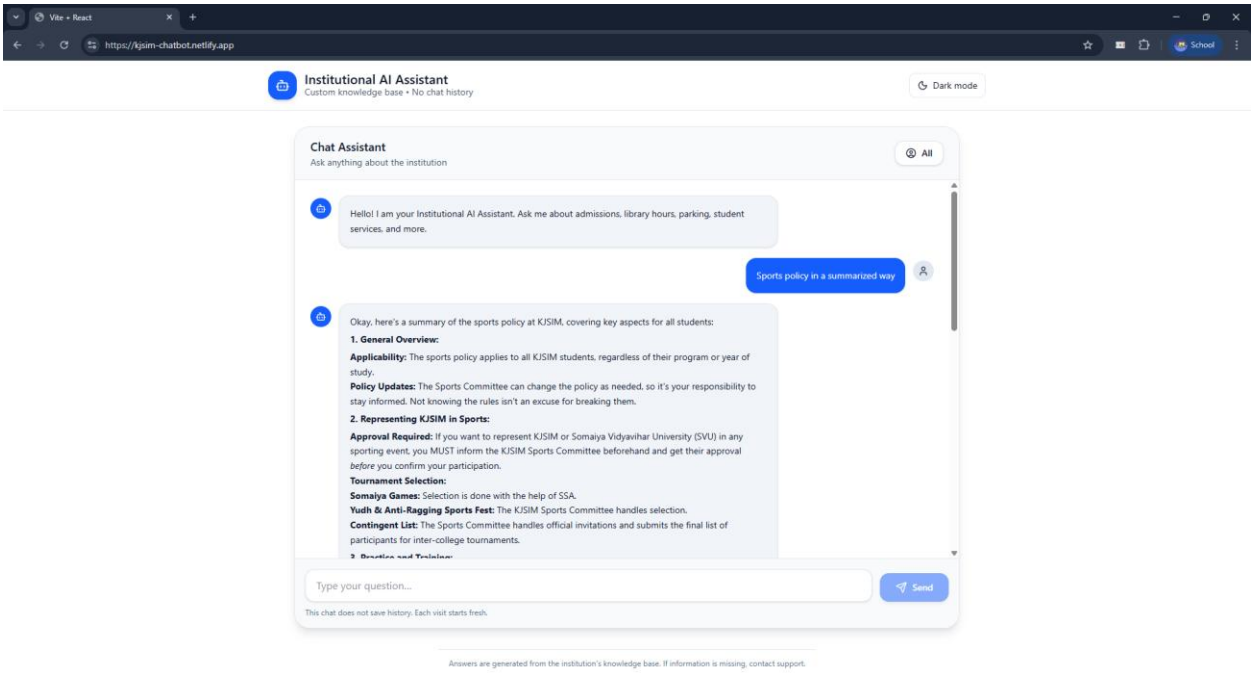


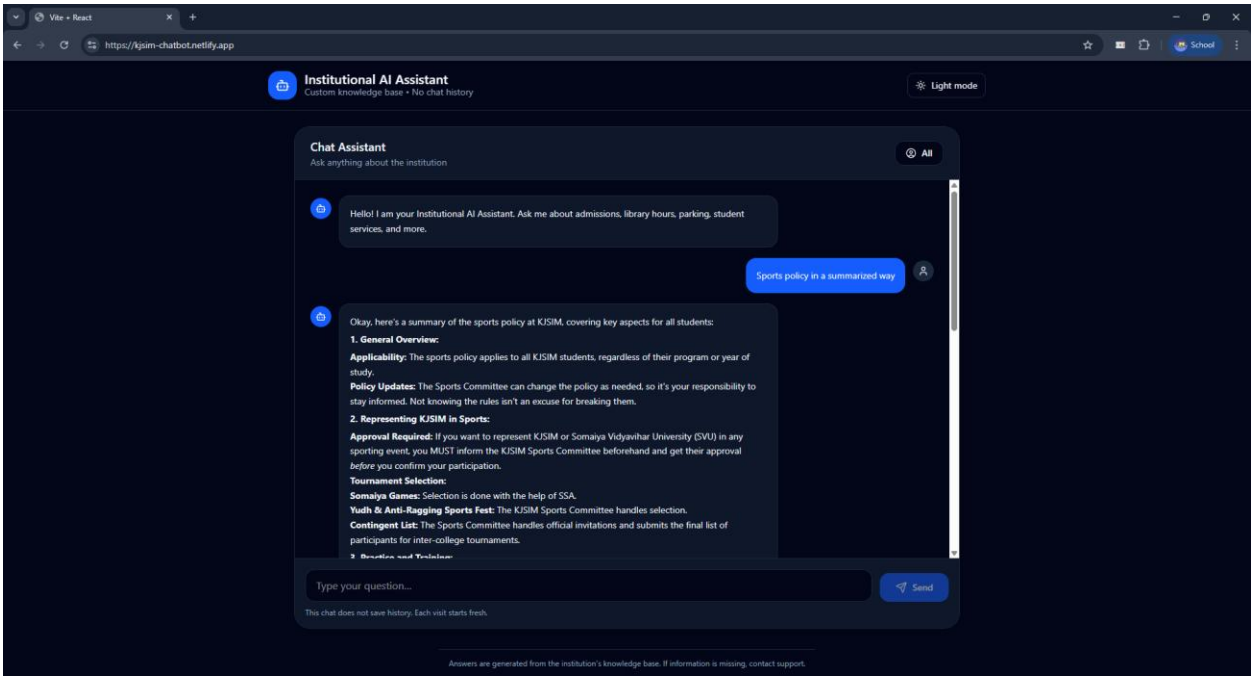
7. User Interface Design

The UI was designed to be clean, modern, and focused. The main components are:

1. **Header:** Displays the chatbot's name and logo.
2. **Message Container:** A scrollable area that displays the conversation history, with user messages and bot messages clearly distinguished.
3. **Input Area:** A text input field with a send button.
   The frontend is built as a Single Page Application (SPA) using React components.




Vite + React
https://kjsim-chatbot.netlify.app
School
Institutional AI Assistant
Custom knowledge base • No chat history
Dark mode
Chat Assistant
Ask anything about the institution
All
Hello! I am your Institutional AI Assistant. Ask me about admissions, library hours, parking, student services, and more.
Sports policy in a summarized way
Okay, here's a summary of the sports policy at KJSIM, covering key aspects for all students:
1. General Overview:
Applicability: The sports policy applies to all KJSIM students, regardless of their program or year of study.
Policy Updates: The Sports Committee can change the policy as needed, so it's your responsibility to stay informed. Not knowing the rules isn't an excuse for breaking them.
2. Representing KJSIM in Sports:
Approval Required: If you want to represent KJSIM or Somaiya Vidyavihar University (SVU) in any sporting event, you MUST inform the KJSIM Sports Committee beforehand and get their approval before you confirm your participation.
Tournament Selection:
Somaiya Games: Selection is done with the help of SSA.
Yudh & Anti-Ragging Sports Fest: The KJSIM Sports Committee handles selection.
Contingent List: The Sports Committee handles official invitations and submits the final list of participants for inter-college tournaments.
Type your question...
Send
This chat does not save history. Each visit starts fresh.
Answers are generated from the institution's knowledge base. If information is missing, contact support.


Vite + React
https://kjsim-chatbot.netlify.app
School
Institutional AI Assistant
Custom knowledge base • No chat history
Light mode
Chat Assistant
Ask anything about the institution
All
Hello! I am your Institutional AI Assistant. Ask me about admissions, library hours, parking, student services, and more.
Sports policy in a summarized way
Okay, here's a summary of the sports policy at KJSIM, covering key aspects for all students:
1. General Overview:
Applicability: The sports policy applies to all KJSIM students, regardless of their program or year of study.
Policy Updates: The Sports Committee can change the policy as needed, so it's your responsibility to stay informed. Not knowing the rules isn't an excuse for breaking them.
2. Representing KJSIM in Sports:
Approval Required: If you want to represent KJSIM or Somaiya Vidyavihar University (SVU) in any sporting event, you MUST inform the KJSIM Sports Committee beforehand and get their approval before you confirm your participation.
Tournament Selection:
Somaiya Games: Selection is done with the help of SSA.
Yudh & Anti-Ragging Sports Fest: The KJSIM Sports Committee handles selection.
Contingent List: The Sports Committee handles official invitations and submits the final list of participants for inter-college tournaments.
Type your question...
Send
This chat does not save history. Each visit starts fresh.
Answers are generated from the institution's knowledge base. If information is missing, contact support.

Development & Implementation

1. Overview

The project was implemented as a full-stack JavaScript application. The frontend serves the user interface, while the backend handles the core AI and business logic. They communicate via a RESTful API.

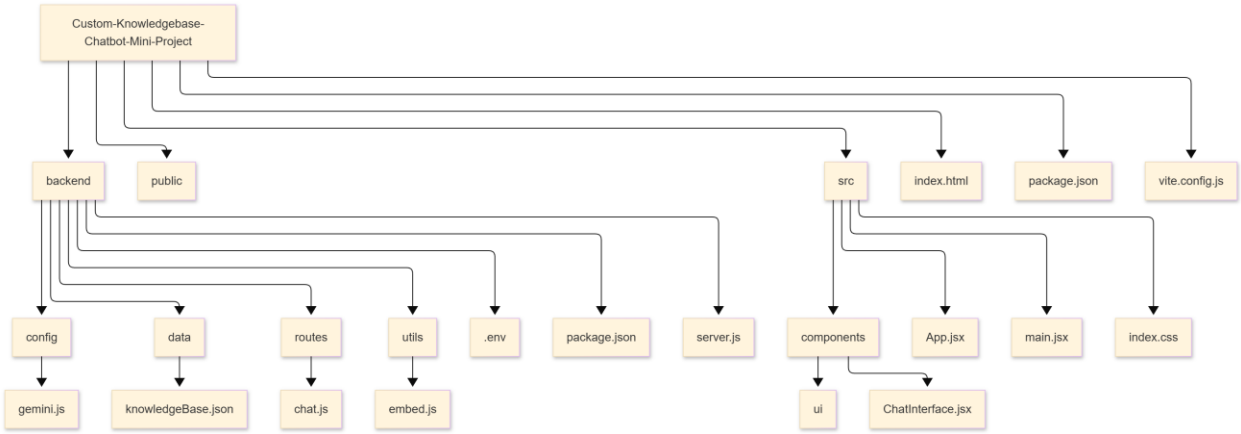
2. High-level Architecture

As described in Section 9.1 and visualized in Figure 1.1.

3. Tech stack

- **Frontend:** Vite, React, Tailwind CSS, Shadcn/ui, Axios.
- **Backend:** Node.js, Express, Google Generative AI SDK, CORS.

4. Folder structure (project)



5. Backend implementation (server.js) — details

The backend is the core of the application. The main server file (server.js) sets up the Express application, middleware (CORS, JSON parsing), and mounts the route handlers.

- **Key Dependencies:** express, @google/generative-ai, cors, dotenv.
- **Flow:**
    1. Loads environment variables.
    2. Configures the Gemini AI client.
    3. Loads the pre-computed knowledge base with embeddings into memory.
    4. Starts the HTTP server listening on a specified port (e.g., 3000).

6. API Design (Backend Routes)

The backend exposes one critical endpoint:

Endpoint	Method	Request Body	Response	Purpose
/chat	POST	{ "message": "User's query string" }	{ text: "Streaming LLM response..." }	Receives user query, executes the RAG pipeline, and streams the answer back.

Implementation of /chat Route (routes/chat.js):

1. **Extract Query:** Gets the user's message from the request body.
2. **Generate Query Embedding:** Calls the generateEmbedding utility function which uses the text-embedding-004 model to convert the query string into a vector.
3. **Semantic Search:** Calculates the cosine similarity between the query vector and every vector in the pre-loaded knowledge base array. Sorts the results and selects the top 1-3 most relevant chunks.
4. **Construct Prompt:** Combines the retrieved context chunks and the original user query into a precise prompt for the LLM. Example:
```
Context:
${context1}
${context2}

Question: ${userQuery}

Answer based on the provided context:
```
5. Generate Response: Sends the constructed prompt to the gemini-1.5-flash model with a streaming configuration.

6. Stream Response: Pipes the streamed response from Gemini back to the frontend client, allowing for a responsive, real-time typing effect.

7. Frontend Implementation (src/App.jsx, src/components/ChatInterface.jsx)

The frontend is a single-page application built with React.

- **State Management:** Uses React's useState hook to manage the list of messages and the current input text.
- **API Communication:** Uses Axios to make a POST request to the backend's /chat endpoint. It handles the streamed response by appending chunks of text to the latest bot message, creating a real-time effect.
- **UI Components:** Leverages Shadcn/ui components like Button, Input, and Avatar to build a polished interface. Styling is done with Tailwind CSS utility classes.
- **Key Features:**
  - Auto-scrolling chat window.
  - Distinct styling for user and bot messages.
  - A "Send" button that is disabled while a request is in flight.

8. Database / PersistenceHow to run locally

- **Knowledge Base:** The knowledgeBase.json file is read into memory when the backend starts. It contains an array of objects, each with id, title, content, and embedding.
- **Chat History:** Persisted only in the frontend's React state for the duration of the browser session.

9. API Examples

Request:

```
curl -X POST http://localhost:3000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "What is the eligibility criteria for sports?"}'
```

Response (Streamed):

```
{"text": "To be eligible..."}
{"text": " students must..."}
...
```

10. How to Run Locally

1. **Clone the repository:** git clone https://github.com/Mohammedvaraliya/Custom-Knowledgebase-Chatbot-Mini-Project
2. **Setup Backend:**
   - cd backend
   - npm install
   - Create a .env file and add GOOGLE_API_KEY=your_actual_api_key_here
   - node server.js (Server runs on http://localhost:3000)
3. **Setup Frontend:**
   - Open a new terminal in the root folder (/Custom-Knowledgebase...).
   - npm install
   - npm run dev (Vite dev server runs on e.g., http://localhost:5173)
4. **Access:** Open http://localhost:5173 in your browser.

11. Deployment

The application can be deployed by:

1. Building the frontend: npm run build (creates a dist folder).
2. Deploying the dist folder to a static host (Vercel, Netlify).
3. Deploying the backend to a Node.js compatible host (Railway, Render, Heroku).
4. Updating the API base URL in the frontend code to point to the live backend.

12. Security & Rate Limits

- The Google API key is kept secure in the backend environment variables.
- The backend is configured with CORS to only accept requests from the designated frontend origin.
- Being a prototype, no explicit rate limiting is implemented, but it is a critical addition for a production system to prevent abuse.

13. Performance & Caching

- Performance is heavily dependent on the Gemini API's latency.
- The in-memory knowledge base provides fast vector search for a dataset of this size.
- The query embedding is generated on every request. For production, caching frequently used query embeddings could improve performance.

14. Implementation Decisions & Notable Tradeoffs

- **In-Memory KB vs. Vector DB:** Chose in-memory for simplicity and cost for a small, static dataset. Tradeoff: not scalable for large, frequently updated knowledge bases.
- **Gemini Flash vs. Pro:** Chose gemini-1.5-flash for its speed and lower cost, which is sufficient for this Q&A task. Tradeoff: gemini-1.5-pro might handle more complex reasoning better.
- **Streaming vs. Single Response:** Implemented streaming for a superior user experience, making the bot feel more responsive.

15. Testing

A multi-faceted testing strategy was employed to ensure the reliability and quality of the system.

Unit Testing

- **Backend:** Tested individual utility functions like the embedding generator and the cosine similarity calculator with mock inputs to verify correct output.
- **Frontend:** Tested React components in isolation to ensure they render correctly with given props (e.g., the Message component renders the correct text).

Integration Testing

- **API Testing:** Used Postman to send HTTP POST requests to the /chat endpoint with various queries. Verified that the response status was 200 and that the streamed content was relevant and well-formed.
- **RAG Pipeline Testing:** Tested the entire backend flow by providing a query and manually inspecting the retrieved context and the final generated answer to ensure it was grounded in the policy document.

Functional/System Testing

Test cases were executed to validate all functional requirements:

Test Case	Input	Expected Output	Result
User submits a valid query about eligibility.	"Who can participate in sports?"	A coherent answer derived from the 'Eligibility' section of the policy.	Pass
User submits a vague query.	"Tell me about sports."	A general answer summarizing key aspects of the sports policy.	Pass
User submits a query with keywords not in the document but with similar meaning.	"What are the rules for using the gym equipment?"	An answer based on the 'Facilities and Equipment' section.	Pass
User submits an out-of-scope query.	"What is the capital of France?"	A polite response indicating inability to answer, or an answer based only on context if found.	Pass

User Acceptance Testing (UAT)

The application was demonstrated to a small group of potential users (peers acting as students). Feedback was collected on the UI/UX, response accuracy, and speed, which was positive and confirmed that the system met its core objectives.

Conclusion

The "Custom Knowledgebase Chatbot for KJSIM" project was successfully designed, implemented, and tested. The developed system effectively addresses the problem of inefficient information retrieval from the institute's sports policy document. By leveraging the RAG architecture with Google's Gemini AI models, the chatbot provides accurate, context-aware, and instantaneous answers to user queries in a natural, conversational manner.

The project demonstrates a practical and powerful application of modern AI technologies—LLMs, vector embeddings, and semantic search—to solve a real-world problem in the educational domain. The use of a robust tech stack (React, Node.js, Express) ensured the development of a scalable and maintainable full-stack web application.

This project serves as a foundational prototype that can be extended with more features, integrated with larger college knowledge bases, and deployed to serve the entire student community, thereby enhancing digital accessibility and operational efficiency at KJ Somaiya Institute of Management.

Limitations

While the project is functional and effective, it has certain limitations that provide avenues for future work:

1. **Static Knowledge Base:** The knowledge base is static and loaded from a JSON file. Any updates to the sports policy require manual reprocessing of the document and redeployment of the backend.
2. **Single Domain:** The chatbot is currently trained only on the sports policy. It cannot answer questions about other college-related topics like academics, admissions, or fees.
3. **Scalability of Vector Search:** The in-memory cosine similarity calculation, while fast for a few hundred chunks, would become a performance bottleneck if the knowledge base grows to thousands or millions of chunks. A dedicated vector database is needed for scaling.
4. **Lack of User Sessions:** There is no persistent user memory across sessions. The chatbot does not "remember" previous interactions if the browser tab is closed.
5. **No Authentication:** The application is open to anyone with the URL. There is no mechanism to restrict access to only KJSIM students/staff.
6. **Model Dependency:** The chatbot's performance is directly tied to the Gemini API's availability, latency, and pricing. Any changes or outages on Google's end would directly impact this service.
7. **Limited Error Handling:** While basic errors are handled, the system could be more robust in dealing with malformed user inputs or upstream API failures.

References

1. Google AI for Developers - Gemini API Documentation: https://ai.google.dev/
2. React Documentation: https://react.dev/
3. Express.js Guide: https://expressjs.com/
4. Tailwind CSS Documentation: https://tailwindcss.com/
5. Vite Documentation: https://vitejs.dev/
6. Shadcn/ui Documentation: https://ui.shadcn.com/
7. Project Repository: https://github.com/Mohammedvaraliya/Custom-Knowledgebase-Chatbot-Mini-Project

12. Glossary

Term	Definition
API	Application Programming Interface. A set of rules that allows programs to talk to each other.
Embedding	A numerical representation of text that captures its semantic meaning, allowing computers to understand the relationship between words and phrases.
Frontend	The part of a web application the user interacts with directly, typically running in a web browser.
Backend	The server-side part of an application responsible for business logic, database interactions, and processing requests from the frontend.
LLM	Large Language Model. A deep learning model trained on vast amounts of text to understand and generate human-like language.
Node.js	A JavaScript runtime environment that executes JavaScript code outside of a web browser, commonly used for building server-side applications.
NLP	Natural Language Processing. A field of AI that gives machines the ability to read, understand, and derive meaning from human languages.
RAG	Retrieval-Augmented Generation. An AI framework that improves the quality of LLM responses by retrieving facts from an external knowledge base.
React	A popular JavaScript library for building interactive user interfaces based on reusable components.
Semantic Search	A search technique that focuses on the meaning of the query and the content, rather than just matching keywords.
Vector Database	A database optimized for storing and querying high-dimensional vector embeddings.

13. Report of Communication with Internal Mentor

Mentor Details

- **Name of Mentor:** Prof. Sudarshan Sirsat
- **Department:** MCA Department
- **Institution:** KJ Somaiya Institute of Management

Date	Mode of Communication	Agenda / Topics Discussed	Outcome / Guidance Provided
15 Sept 2025	In-Person Meeting	Project ideation and initial proposal. Discussion on feasibility of a custom chatbot for a specific domain.	Mentor approved the topic. Suggested focusing on a well-defined, limited-scope document (like the sports policy) for the MVP.
01 Oct 2025	Email Update	Shared initial progress on tech stack selection (MERN vs PERN) and data preprocessing.	Advisor recommended proceeding with the MERN-like stack (using Gemini instead of OpenAI) and emphasized the importance of a clean UI.
20 Oct 2025	In-Person Meeting & Demo	Demonstration of the first working prototype with a basic RAG pipeline.	Mentor provided positive feedback. Suggested implementing response streaming for a better UX and advised starting the project report early.

Date	Mode of Communication	Agenda / Topics Discussed	Outcome / Guidance Provided
05 Nov 2025	WhatsApp Communication	Discussed a challenge with CORS configuration when connecting the frontend to the backend.	Advisor guided on correctly setting up the CORS middleware in the Express server to allow requests from the Vite dev server.
20 Nov 2025	Final Review Meeting	Presented the finalized application, including the polished UI, full RAG implementation, and testing results.	Mentor conducted UAT and confirmed that all core objectives were met. Provided final suggestions for the project report structure and content.